



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SMART EV CHARGING STATION QUEUE AND SLOT SCHEDULING SYSTEM

*A Capstone Project Report submitted in partial fulfilment of the requirements for the
Course of*

CSA0604-Design and Analysis of Algorithms

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

AIML

Submitted by

K Baba Satya Kumar (192525226)

Under the Supervision of

Dr M Manikandan

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Smart Ev Charging Station Queue And Slot Scheduling System ” has been carried out K Baba Satya Kumar (192525226) under the supervision of Dr M Manikandan and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech AIML Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. F. Mary Harin Fernandez
Professor,
Department of CSE
SIMATS Engineering,
SIMATS,Chennai 602105

COURSE FACULTY

Dr M Manikandan
Professor
Department of CSE
SIMATS Engineering,
SIMATS,Chennai 602105



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DECLARATION

We, K Baba Satya Kumar (192525226) of the Department of Artificial Intelligence and Machine Learning SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Smart Ev Charging Station Queue And Slot Scheduling System” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Student with Names

K.Baba Satya Kumar (192525226)

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
K Baba Satya Kumar 192525226	Module 1 – User & Vehicle Management; user, vehicle, validation and persistence design	User/vehicle data model, interface flow, validation and database integration	Planned functional and integration tests for Module 1	Ch. 1–5 sections related to Module 1	25%
K. Anil Kumar 192525231	Module 2 – Charging Station Management	Charging-station information and availability management	Planned station and availability tests	Module 2 contribution	25%
N. Deepak 192572075	Module 3 – Queue & Slot Scheduling	Queue and slot scheduling workflow	Planned queue/slot tests	Module 3 contribution	25%
G. Hari Prasad 192525273	Module 4 – Dashboard & Reports	Dashboard and report presentation	Planned dashboard/report verification	Module 4 contribution	25%
Total					100%

ABSTRACT

Electric vehicle charging is increasingly treated as an infrastructure and service-management problem in which limited charging resources must be coordinated against competing user requests. The proposed Smart EV Charging Station Queue and Slot Scheduling System addresses the information-management layer of this problem through a web-based application architecture using Python, Flask, HTML5, CSS3, JavaScript where required, and SQLite/SQL. The overall design is divided into four interacting modules: User & Vehicle Management, Charging Station Management, Queue & Slot Scheduling, and Dashboard & Reports. This individual report concentrates on Module 1 – User & Vehicle Management, covering user registration, login and authentication concepts, profile management, input validation, vehicle registration, vehicle update/deletion, user–vehicle relationships, persistence, error handling, and integration with charging requests. At system level, the scheduling model follows a deterministic sequence: a charging request is associated with a validated user and vehicle, slot availability is checked, an available slot is allocated, and an unavailable request is entered into a queue whose order and status are maintained. The design deliberately separates application presentation, validation/business logic, and database access. Charging standards such as IEC 61851-1 and OCPP are reviewed as reference contexts; no physical charger control or protocol implementation is claimed in the present scope. Because this report documents the design and implementation plan before measured system execution, it does not claim execution times, accuracy values, database counts, user counts, station counts, or performance measurements. The intended engineering outcome is a structured academic prototype that demonstrates controlled identity and vehicle data management and provides a sound software foundation for coordinated charging-slot allocation and queue handling.

Keywords: Electric Vehicle Charging, Queue Management, Slot Scheduling, Flask, SQLite, User–Vehicle Management

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS / SYMBOLS	viii
1	CHAPTER 1: Introduction and Engineering Problem	1
2	CHAPTER 2: Literature Review and Existing Solutions	3
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	4
4	CHAPTER 4: Implementation, Testing & Results, Project Management	8
5	CHAPTER 5: Conclusion, Future Work and Reflection	11
	References	12
	Appendices	13
	Capstone Project Outcome Mapping Sheet	14

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1	Overall System Architecture	5
Figure 2	Module 1 – User & Vehicle Management Architecture	5
Figure 3	Module 1 – User & Vehicle Management Flowchart	6
Figure 4	Conceptual Database / ER-style Model	6
Figure 5	Queue & Slot Scheduling Workflow	6
Figure 6	Complete System Workflow	7
Figure 7	Planned Development Timeline	9

All figures are conceptual engineering designs for the proposed software prototype. They describe intended interfaces and data/control relationships rather than claiming completed runtime evidence.

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Engineering Problem Factors	1
Table 2	Literature / Existing Solution Comparison	3
Table 3	Functional and Non-Functional Requirements	4
Table 4	Constraints and Applicable Standards	4
Table 5	Design Specifications	4
Table 6	Alternative Solution Evaluation	5
Table 7	Sustainability, Ethics, Safety and Security	7
Table 8	Risk Register	7
Table 9	Development Tools and Resources	8
Table 10	Test Plan and Verification Matrix	9
Table 11	Objective Achievement Matrix	9
Table 12	Project Planning and Milestones	10
Table 13	Planned Budget Structure	10
Table 14	Module Responsibility and Integration Matrix	10
Table 15	Project Management Control Points	10

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
EV	Electric Vehicle	—
EVSE	Electric Vehicle Supply Equipment	—
OCPP	Open Charge Point Protocol	—
CS	Charging Station Management System	—
UI	User Interface	—
DB	Database	—
SQL	Structured Query Language	—

API	Application Programming Interface	—
IEC	International Electrotechnical Commission	—
AC	Alternating Current	V
DC	Direct Current	V
ID	Identifier	—

Notation used in the report: “planned” describes intended functionality; “measured result” is reserved for values produced by executed tests. Standards and protocols are discussed as engineering context unless an implementation is explicitly described.

CHAPTER 1 INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Electric vehicle charging is increasingly viewed as a combined infrastructure and service-management challenge. A charging location can have a finite number of chargeable spaces or connector resources while demand may arrive in overlapping time windows. The software layer must therefore represent users, vehicles, charging resources and requests in a consistent way so that decisions about availability and waiting state are traceable. The International Energy Agency reported strong growth in public charging infrastructure in its Global EV Outlook 2025, reinforcing the practical relevance of organized charging-resource management.

The proposed project is a Smart EV Charging Station Queue and Slot Scheduling System. It is designed as a web-based academic prototype using Python, Flask, HTML5, CSS3, JavaScript where needed, and SQLite/SQL. The present scope focuses on the information and scheduling layer rather than direct physical control of charging equipment. The individual contribution is Module 1 – User & Vehicle Management, which establishes the identity and vehicle data required before a charging request can be processed.

1.2 Need for the Project

A station with limited charging resources must make allocation decisions when several users request service at the same time. Manual coordination can create ambiguity about which resource is available, whether a vehicle belongs to a registered user, and whether an unavailable request is waiting or rejected. A centralized application reduces these ambiguities by maintaining a single logical representation of users, vehicles, requests, slots and states.

Factor	Engineering implication	Relevance
Limited charging points	Simultaneous requests cannot all be served immediately.	Requires slot availability tracking.
Overlapping requests	Resource conflicts can occur when allocation is not centralized.	Requires request status and allocation rules.
User / vehicle data	A charging request must reference a valid user and vehicle.	Requires validation and database relationships.
Changing slot status	An allocated resource must not remain marked available.	Requires controlled status updates.
Waiting requests	Unavailable resources need a transparent waiting mechanism.	Requires queue insertion and status tracking.
Operational visibility	Users and operators need current request and resource information.	Supports dashboard and report functions.

Table 1. Engineering Problem Factors

1.3 Problem Statement

Design a structured software system that manages users, vehicles, charging stations, charging slots, and charging requests when available charging resources are limited and requests may overlap. The system shall preserve data integrity, validate inputs, associate vehicles with users, allocate an available slot where possible, place an unavailable request into a queue, and maintain status information that can be consumed by a dashboard or reporting layer.

1.4 Project Aim

The aim is to design and implement a web-based prototype for organized EV charging-resource management, with the individual contribution focused on reliable User & Vehicle Management and its integration with the charging-request workflow.

1.5 Project Objectives

1. Design a user and vehicle information model suitable for charging-service requests.
2. Develop user registration, authentication concepts, profile management and validation flows.
3. Develop vehicle registration, update, deletion and information-management functions.
4. Establish logical user–vehicle relationships and persistent storage using SQLite/SQL.
5. Define a queue-based charging-request workflow for limited slots.
6. Prepare functional and integration tests for user, vehicle, request, queue, allocation and status functions.

1.6 Scope

Included: user and vehicle records, validation, database persistence, charging-station and slot information model, request intake, queue logic, slot status representation, dashboard/report information model and planned functional testing. Excluded: physical charger control, payment processing, real-time charger telemetry, OCPP communication, cloud deployment, IoT integration and machine-learning optimization. Assumptions: the prototype uses a defined logical representation of charging slots and operates on application-level data. Boundary conditions: performance metrics and runtime evidence are not part of the design-only baseline.

1.7 Engineering Constraints

The principal constraints are limited software development resources, the need for consistent database state, variability in user input, the need for deterministic scheduling logic and the requirement that the application remain auditable. The design also has an evidence constraint: claims about actual execution, throughput, accuracy, user count, station count or cost are reserved for the implementation/testing stage.

1.8 Proposed Technical Approach

The application is planned around a Flask web layer, HTML5/CSS3/JavaScript interfaces and SQLite/SQL persistence. Presentation, validation/business logic and data access are treated as separate responsibilities. A charging request references a validated user and vehicle; slot availability is then evaluated. An available resource is allocated, while an unavailable request is inserted into the queue. Status transitions are persisted so that the dashboard/report layer can present the current state.

1.9 Expected Engineering Outcome

The intended outcome is a demonstrable academic prototype with traceable user and vehicle records, structured charging requests, explicit slot status, deterministic queue handling and a modular software architecture. Module 1 is expected to provide the clean identity/data foundation on which the remaining modules can operate.

CHAPTER 2 LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and technical resources were identified from recognized energy and standards organizations, official protocol documentation, and software documentation. The review emphasizes charging infrastructure growth, charging-management functions, queue/resource scheduling ideas, digital booking patterns and interoperability standards. The purpose is to define the engineering context and the scope boundary of the academic prototype rather than to claim protocol certification or industry-scale deployment.

2.2 Review of Existing Work

The literature indicates that EV charging management extends beyond the electrical connection itself. Public charging systems must represent physical resources, communicate status, manage user interaction and coordinate availability. In parallel, generic queueing systems provide a deterministic mechanism for handling demand when service capacity is temporarily constrained. Digital booking systems contribute reservation interfaces, record persistence and status tracking, while industry interoperability standards define communication behavior between charging equipment and management systems.

The proposed project combines these ideas at the software information layer. It does not attempt to reproduce the full feature set of a commercial charging platform. Instead, it provides a compact academic prototype in which a valid user and registered vehicle form the identity foundation for a charging request, while limited slots are handled by allocation or queue insertion.

Existing approach / technology	Advantages	Limitations for this project	Relevance
Manual / spreadsheet scheduling	Simple to start and requires little infrastructure.	Weak concurrency control, validation and status consistency.	Motivates centralized software workflow.
Commercial charging management platform	Handles real operational resources, user services and broader integration.	Beyond academic prototype scope and may include proprietary features.	Provides context for required resource/status concepts.
Web booking prototype	Accessible interface and structured resource records.	May not represent real charger telemetry or electrical constraints.	Closest architectural basis for the proposed prototype.
OCPP-based management	Supports standardized charger-to-system communication.	Protocol integration is outside the current implementation scope.	Reviewed as interoperability context only.
Proposed system	Combines user/vehicle records with slot allocation and queue status.	Prototype scope; no physical charger control.	Directly addresses the defined academic information-management problem.

Table 2. Literature / Existing Solution Comparison

2.3 Comparative Analysis

The comparison supports a deliberate scope boundary. Commercial systems and protocol-based platforms solve broader operational and interoperability problems, whereas the proposed academic system concentrates on identity, resource representation and queue/slot status. This narrower focus supports explainable software design and makes the individual Module 1 contribution explicit.

2.4 Engineering Gap

A practical gap exists between simple digital booking and a charging-oriented resource-management prototype that explicitly connects validated user and vehicle information to charging requests and waiting-state management. The project does not claim to invent a new scheduling theory or charging protocol. Its engineering contribution is an integrated software model that links identity, vehicle association, resource availability and queue state in one auditable workflow.

2.5 Proposed Contribution

The individual contribution is the User & Vehicle Management module. Its role is to guarantee that downstream charging operations receive validated records. The dependency chain is: a valid user registers or selects a valid vehicle; the charging request references that vehicle; station and slot management evaluate the request; and the outcome is reflected in allocation or queue status.

CHAPTER 3 ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

The requirements are separated into functional and non-functional requirements. Each target is a design requirement to be validated during implementation; it is not a claim that the target has already been achieved.

Type	Requirement	Measurable target / verification method
Functional	Register user	Required fields stored; invalid or conflicting input rejected.
Functional	Authenticate user	Valid credentials accepted; invalid credentials rejected.
Functional	Manage profile	Permitted profile fields can be viewed and updated.
Functional	Register vehicle	Vehicle record linked to a valid user.
Functional	Update / delete vehicle	Authorized change persists and relationship rules are respected.
Functional	Validate inputs	Malformed, missing or inconsistent values produce controlled feedback.
Functional	Charging request integration	Request references a valid user and vehicle.
Functional	Queue unavailable request	Unavailable slot results in a queue state.
Functional	Allocate available slot	Available slot is assigned and status updated.
Non-functional	Data consistency	User, vehicle, request and slot states remain logically consistent.
Non-functional	Usability	Common operations reachable through clear forms/workflows.
Non-functional	Maintainability	Presentation, business logic and data access remain modular.

Table 3. Functional and Non-Functional Requirements

3.2 Constraints & Applicable Standards

Constraint / Standard	Requirement	How applied
IEC 61851-1:2017	General requirements and safety context for conductive EV charging systems.	Used as reference context; physical EVSE control is excluded.
OCPP 2.0.1 / 2.1	Open communication between charging stations and management systems.	Reviewed as interoperability context; not implemented or certified.
SQLite / SQL	Persistent relational storage for prototype records.	Planned as the application database layer.
Academic evidence discipline	Engineering claims should be supported by executed evidence.	Design-stage statements are separated from measured outcomes.

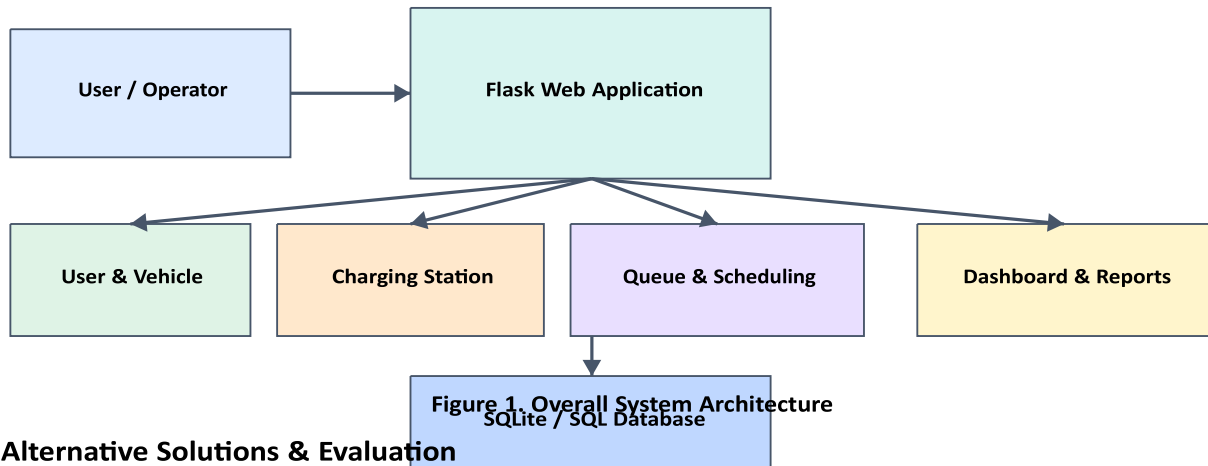
Table 4. Constraints and Applicable Standards

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Application framework	Flask-based web application	—	High
Persistence	SQLite / SQL relational database	—	High
User identity	Unique user identifier and validated profile fields	record	High
Vehicle association	Each vehicle linked to a valid user	relationship	High

Slot state	Availability represented explicitly	state	High
Queue state	Unavailable requests stored with status/order	state	High
Input handling	Validation and controlled error feedback	—	High
Physical charger control	Excluded from current scope	—	Excluded

Table 5. Design Specifications



3.4 Alternative Solutions & Evaluation

Alternative 1: Manual scheduling is easy to start but provides limited automation and weak state consistency. Alternative 2: Booking-only software improves record keeping but may not represent an explicit waiting queue. Alternative 3: The proposed modular Flask prototype combines identity, vehicle records, resource status, request handling and queue logic.

Criterion	Weight	Manual scheduling	Booking-only	Proposed modular prototype
Performance	25%	Low	Medium	Medium-High
Data consistency	20%	Low	Medium	High
Usability	20%	Medium	High	High
Scalability	15%	Low	Medium	Medium
Traceability	10%	Low	Medium	High
Scope fit	10%	Medium	High	High

Table 6. Alternative Solution Evaluation

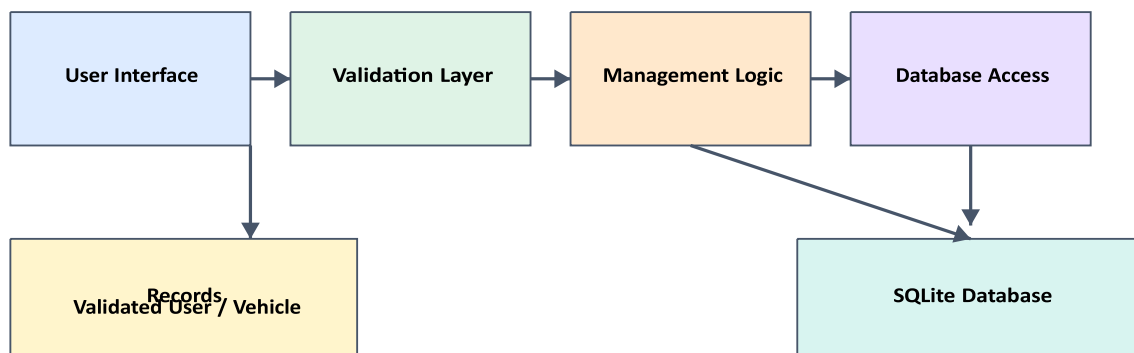


Figure 2. Module 1 – User & Vehicle Management Architecture

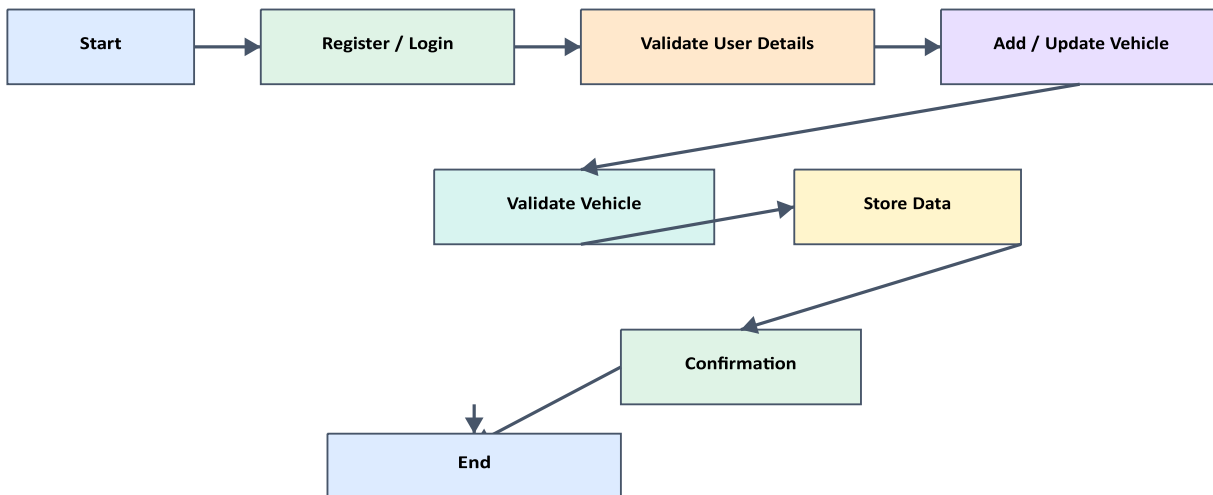


Figure 3. Module 1 – User & Vehicle Management Flowchart

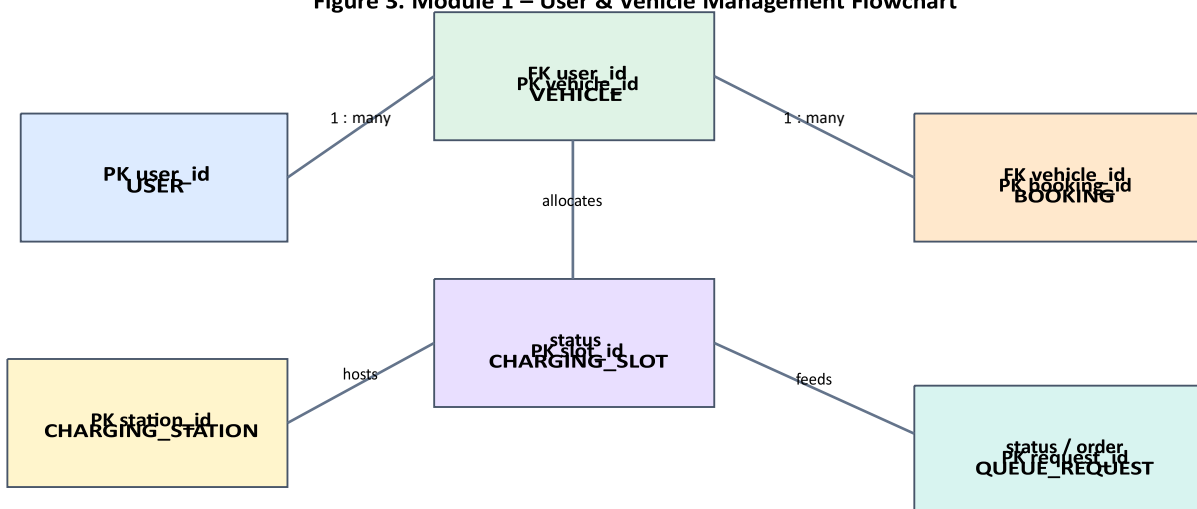


Figure 4. Conceptual Database / ER-style Model

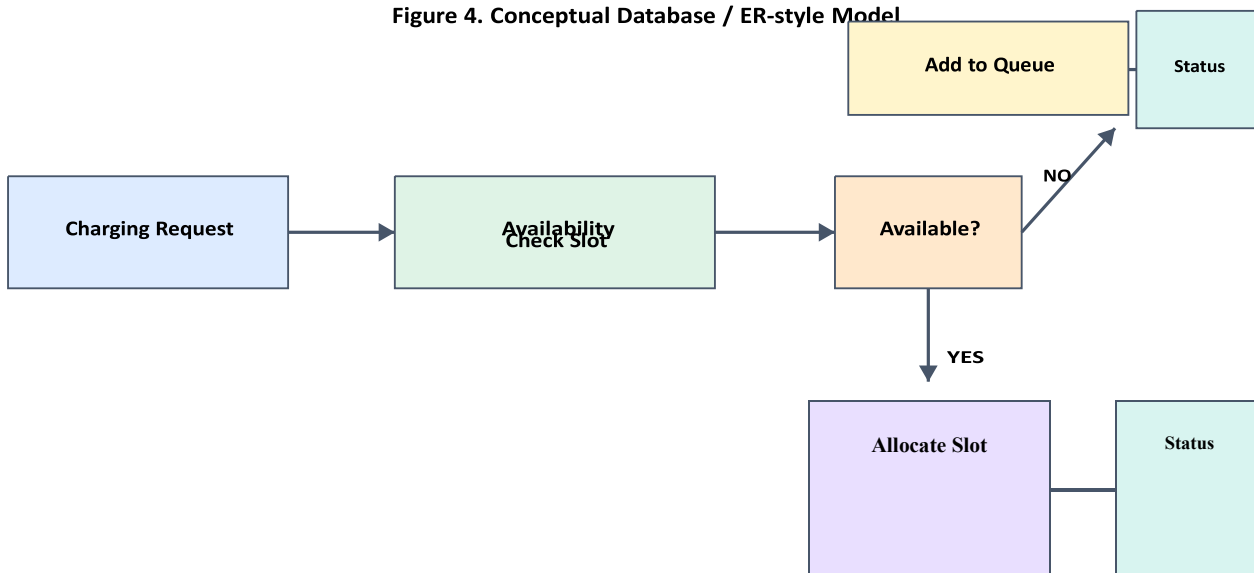
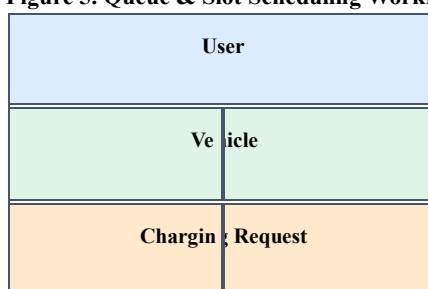


Figure 5. Queue & Slot Scheduling Workflow



Station / Slot Check	
Slot Allocation / Queue	
Status Update	
Dashboard / Report	

Figure 6. Complete System Workflow

3.5 Selected Design & Detailed Design

The proposed modular Flask architecture is selected because it offers a balanced combination of implementation simplicity, separation of responsibilities, local persistence and explicit control over state transitions. The decision follows the evaluation matrix: the design directly addresses the identified engineering factors without introducing unsupported protocol or hardware claims.

3.6 Trade-off Analysis

The central trade-off is between feature breadth and verifiability. Adding real charger telemetry, payments, mobile messaging or protocol integration would increase functionality but would also introduce additional interfaces, security boundaries and external dependencies. The current design deliberately keeps the application layer narrow so that Module 1 and the queue/slot workflow can be implemented, inspected and tested independently.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence evaluated	Impact on final decision
Sustainability	Prefer digital coordination over paper/manual records.	Electronic persistence and status tracking.	Supports a software-first architecture.
Ethics	Protect user identity and vehicle-related information.	Need-to-know data handling and validation.	Encourages controlled access and minimal data exposure.
Health & Safety	Avoid representing software as direct physical charger control.	Scope boundary and IEC context.	Keeps electrical safety outside unsupported software claims.
Security	Protect accounts and input paths.	Authentication concept, validation and database constraints.	Makes validation and authorization explicit design requirements.

Table 7. Sustainability, Ethics, Safety and Security

Risk	Likelihood	Severity	Risk level	Mitigation	Residual risk
Invalid user input	Medium	Medium	Medium	Server-side validation and controlled error handling.	Low–Medium
Duplicate / inconsistent vehicle data	Medium	Medium	Medium	Unique identifiers and user–vehicle relationship constraints.	Low
Incorrect slot state	Medium	High	High	Single status transition path and transaction-aware updates.	Medium
Unauthorized account access	Medium	High	High	Credential checks, session control and authorization rules.	Medium
Queue ordering defect	Low–Medium	Medium	Medium	Explicit queue order field and test cases.	Low–Medium

Table 8. Risk Register

3.8 Planned Module Integration View

The four modules are treated as interacting software subsystems. Module 1 supplies validated user and vehicle records. Module 2 maintains station and slot information. Module 3 consumes a valid request and determines allocation or queue state. Module 4 presents the resulting status for operational visibility.

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01 User registration	Submit valid registration details.	User record is stored and confirmation is shown.	Not measured at pre-implementation stage.	Planned
TC-02 Invalid user input	Submit missing/malformed required fields.	Validation message shown; invalid data not stored.	Not measured at pre-implementation stage.	Planned
TC-03 User update	Edit authorized profile fields.	Updated values persist.	Not measured at pre-implementation stage.	Planned
TC-04 Vehicle registration	Add vehicle for valid user.	Vehicle record linked to user.	Not measured at pre-implementation stage.	Planned
TC-05 Vehicle update	Edit valid vehicle record.	Updated values persist.	Not measured at pre-implementation stage.	Planned
TC-06 Invalid vehicle details	Submit malformed vehicle data.	Validation message shown; invalid data rejected.	Not measured at pre-implementation stage.	Planned
Test Case	Input / Action	Expected Result	Actual Result	Status
TC-07 User–vehicle relationship	Select vehicle for its owning user.	Valid reference accepted.	Not measured at pre-implementation stage.	Planned
TC-08 Charging request	Create request using valid user and vehicle.	Request enters allocation workflow.	Not measured at pre-implementation stage.	Planned
TC-09 Available slot	Submit request with free slot.	Slot is allocated and state updated.	Not measured at pre-implementation stage.	Planned
TC-10 Unavailable slot	Submit request when slot is occupied.	Request enters queue state.	Not measured at pre-implementation stage.	Planned
TC-11 Queue insertion	Inspect unavailable requests.	Queue order/status is retained.	Not measured at pre-implementation stage.	Planned
TC-12 Dashboard update	Process allocation/queue state change.	Dashboard reflects latest status.	Not measured at pre-implementation stage.	Planned

CHAPTER 4 IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The implementation is planned as an incremental web-application workflow. First, the data model for users and vehicles will be established in SQLite. Next, Flask routes and validation logic will be introduced, followed by interface forms for registration and profile/vehicle management. Finally, the module will be linked to the charging-request flow so that only valid user–vehicle combinations can enter allocation or queue processing. Testing will be carried out against functional requirements before the system is treated as an achieved implementation.

Tool / software / resource	Purpose	Stage used
Python	Application programming language and backend logic.	Implementation
Flask	Web routing, request handling and application structure.	Implementation
HTML5	Page structure and form interfaces.	Implementation
CSS3	Presentation and responsive layout.	Implementation
JavaScript	Client-side interaction where required.	Implementation
SQLite / SQL	Persistent relational storage.	Implementation
VS Code	Development, debugging and source management.	Implementation / testing

Table 9. Development Tools and Resources

4.2 Software Implementation

Module 1 is centered on the following logical operations: user registration, login/authentication concept, profile management, vehicle registration, vehicle update/delete, validation, database persistence, error handling and integration with charging requests.

Input validation is expected at both the interface and application layers so that database integrity does not depend solely on browser behavior.

Layer	Planned implementation responsibility
Presentation	Forms for user registration, login, profile and vehicle operations.
Validation / business logic	Field checks, authorization rules, relationship checks and controlled error handling.
Data access	Parameterized SQL operations for insert, update, delete and retrieval.
Integration	Validated user–vehicle reference supplied to charging request processing.
Status flow	Request linked to slot check, allocation or queue state.

Implementation evidence available at this report stage is limited to the proposed architecture and workflow. Runtime screenshots and measured outputs are intentionally not inserted as completed evidence.

4.3 Test Plan & Verification

Table 10. Test Plan and Verification Matrix

4.4 Results

At the pre-implementation stage, the report records design outputs, requirements and a planned test protocol rather than fabricated measurements. The key result of this phase is a complete software architecture and testable workflow: user → vehicle → charging request → station/slot check → allocation or queue → status update → dashboard. Quantitative performance, runtime screenshots and database counts will only become reportable after execution.

4.5 Data Analysis & Engineering Judgment

The principal engineering judgment at this stage is that explicit state representation is more important than adding unsupported automation. A request must have a known owner, a known vehicle and a deterministic outcome. In the proposed model, the state transition is visible and testable, which reduces ambiguity during later implementation. Performance analysis can subsequently be added using measurements such as response time, request throughput and queue waiting time.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence planned / produced	Achievement
User and vehicle information model	Conceptual data model and Module 1 architecture.	Design complete
Registration and validation workflows	Module 1 flowchart and test cases.	Design complete
Persistent storage	SQLite / SQL design and data-access layer plan.	Design complete
Queue-based request workflow	Queue workflow and integrated system flow.	Design complete
Testing and evaluation	Defined test plan and acceptance criteria.	Planned execution

Table 11. Objective Achievement Matrix

4.7 Strengths & Limitations

- Strengths: modular structure, clear user–vehicle relationship, explicit state transitions, local relational persistence, testable workflows and a scope that is appropriate for an academic prototype.
- Limitations: no direct charger control, no live telemetry, no payment integration, no cloud deployment, no OCPP implementation and no measured performance at the design stage.

4.8 Project Planning, Roles & Budget

Planned Development Timeline



Sequence: define → design → build → integrate → test → demonstrate *Actual dates and completion status are recorded during project execution.*

Figure 7. Planned Development Timeline

Project coordination is organized as a sequence of requirements, data modelling, implementation, module integration, testing and final demonstration. Module responsibilities are distributed by the four project modules, while the individual report documents the contribution of K Baba Satya Kumar to Module 1.

Milestone	Main activity	Output
M1	Requirements and problem definition	Scope, objectives and constraints
M2	Database and Module 1 design	User/vehicle schema and validation logic
M3	Flask implementation	Routes, forms and persistence
M4	Module integration	Charging request and queue interface
M5	Testing	Test execution records and defect log
M6	Final report and demonstration	Validated report and prototype demonstration

Table 12. Project Planning and Milestones

Item	Estimated Cost	Actual Cost
Software tools listed in implementation plan	Local / existing academic resources	Not recorded at design stage
Development environment	Existing computer and software environment	Not recorded at design stage
Database	SQLite / SQL software stack	No separate software purchase planned
Deployment	Local prototype	Not recorded at design stage

Table 13. Planned Budget Structure

Module	Primary responsibility	Integration dependency
Module 1 – User & Vehicle Management	Identity, profile, vehicle records, validation and persistence.	Supplies validated user–vehicle data to charging requests.
Module 2 – Charging Station Management	Station information, connector/slot representation and availability state.	Provides resource state for scheduling.
Module 3 – Queue & Slot Scheduling	Allocation, waiting queue and status transitions.	Consumes request and resource state.
Module 4 – Dashboard & Reports	Status presentation and report-oriented views.	Displays allocation, queue and resource information.

Table 14. Module Responsibility and Integration Matrix

Management control point	Evidence to maintain	Decision / action
Scope control	Approved feature list and exclusions	Prevent unsupported features from entering the baseline.
Change control	Change description and reason	Review effect on requirements, interfaces and testing.
Test evidence	Test case ID, input, output and defect note	Accept only results supported by executed evidence.
Data integrity review	Schema, relationship and constraint checks	Correct ownership or state inconsistencies before integration.
Submission readiness	Report, source, screenshots and demonstration checklist	Confirm completeness before final academic submission.

Table 15. Project Management Control Points

CHAPTER 5 CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Smart EV Charging Station Queue and Slot Scheduling System is designed to address a practical software-management problem: limited charging resources may receive multiple overlapping service requests, creating a need for structured identity, availability and queue management. The proposed Flask and SQLite-based architecture separates user and vehicle information management from station, queue and dashboard responsibilities while connecting the modules through well-defined data relationships. The individual Module 1 contribution establishes validated users and vehicles as the information foundation for subsequent charging requests.

The report deliberately distinguishes engineering design from measured implementation. At this stage, the strongest validated outputs are the requirements, architecture, data relationships, workflow and test plan. Once the prototype is executed, the same structure can be extended with actual screenshots, database records, measured response times and test outcomes without changing the fundamental engineering model.

5.2 Future Work

Possible extensions include OCPP integration for standardized charger communication, real charger telemetry, mobile notifications, payment integration, cloud deployment, demand prediction, optimization, IoT integration and more advanced scheduling strategies. These are future engineering directions rather than current features of the academic prototype.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired: application-layer resource management, relational modelling for user–vehicle associations, validation design, deterministic queue workflows, and software-system decomposition.

Engineering & Professional Skills Developed: requirements analysis, modular architecture, data modelling, interface planning, test-case design, documentation and engineering judgment about scope and evidence.

Individual Reflection – K Baba Satya Kumar (Reg. No. 192525226), Module 1 – User & Vehicle Management

The reflection in this pre-implementation report is intentionally written as a structured draft rather than as a claim about personal implementation experience. The most important engineering problem addressed in the assigned module is preserving a reliable relationship between users and their vehicles so that later charging requests cannot reference incomplete or unrelated data. The key design decision is to place validation and persistence before charging-request processing, because downstream modules depend on trustworthy identity data. The independently developed knowledge areas are Flask request handling, relational persistence with SQLite/SQL, input validation and interface-to-database integration. When the implementation is complete, this paragraph should be replaced or expanded with the student’s actual debugging experience, evidence of a personal design decision, and a concrete redesign that would be made in a second iteration.

REFERENCES

- [1] International Energy Agency, Global EV Outlook 2025: Electric Vehicle Charging, Paris: IEA, 2025.
- [2] Open Charge Alliance, “OCPP: Discover our open standards for future-ready EV charging,” Open Charge Alliance, accessed Sep. 4, 2026.
- [3] Open Charge Alliance, “New editions of the OCPP 2.1 and 2.0.1 now available!,” Dec. 4, 2025.
- [4] Open Charge Alliance, “Download OCPP,” Open Charge Alliance, accessed Sep. 4, 2026.
- [5] International Electrotechnical Commission, IEC 61851-1:2017, Electric vehicle conductive charging system – Part 1: General requirements, IEC, 2017.
- [6] Open Charge Alliance, “Certification OCPP 2.0.1,” Open Charge Alliance, accessed Sep. 4, 2026.
- [7] Python Software Foundation, Python Documentation, software documentation resource, accessed Sep. 4, 2026.
- [8] Pallets, Flask Documentation, web framework documentation, accessed Sep. 4, 2026.

APPENDICES

Appendix A – Detailed Calculations

No lengthy numerical calculations are required for the present software design stage. The principal scheduling model is logical: request → availability check → allocation or queue insertion → status update. After implementation, measured waiting time, queue length, throughput or response-time calculations may be added from executed test data.

Appendix B – Engineering Drawings / Schematics

Figures 1–7 in Chapter 3 contain the primary architecture, flow, data and workflow diagrams. Additional implementation-level database schemas or interface wireframes may be attached after the final code structure is frozen.

Appendix C – Source Code / Code Repository Information

The complete application source is maintained separately from the report in the project workspace. This report intentionally contains the architecture and test plan rather than duplicating the full source code. The final approved repository location and commit reference can be inserted at submission time.

Appendix D – Datasheets

No physical charger hardware or component datasheets are claimed for the present software-only prototype scope.

Appendix E – Experimental / Raw Data

Raw test data will be appended after the implementation and test execution stage. The report design reserves this appendix for screenshots, output logs, test measurements and defect observations.

Appendix F – Ethics / Institutional Approval

The present work is a software engineering prototype. Any departmental or institutional approval required for later user studies, data collection or deployment will be documented here.

Appendix G – Project Meeting / Progress Records

Meeting minutes, milestone reviews and progress evidence can be appended here after project execution.

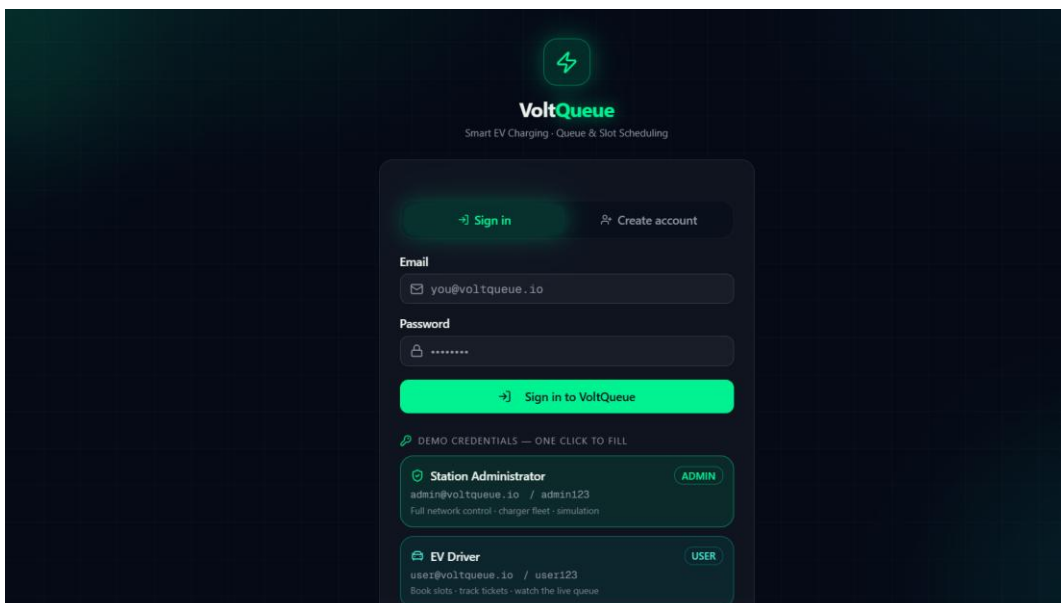
Appendix H – User / Industry Feedback

Feedback can be added after a review or demonstration session.

Appendix I – Publications / Patents / Competitions / Product Outcomes No such outcome is claimed in the present design-stage report.

Appendix J – Individual Contribution Evidence

The signed individual contribution statement and module-specific implementation evidence for K Baba Satya Kumar should be attached here at final submission.



CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome	Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	Problem statement and engineering-factor analysis	SO1	Ch. 1, Ch. 2
Engineering design under constraints	Requirements, standards, alternatives and trade-offs	SO2	Ch. 3
Written/oral technical communication	Structured capstone report and figures	SO3	Whole report + Viva
Ethics and professional responsibility	Sustainability, ethics, safety and security analysis	SO4	Ch. 3 (3.7)
Teamwork and leadership	Module allocation and project planning	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	Test plan, acceptance criteria and analysis framework	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	Reflection on software, database and workflow design	SO7	Ch. 5 (5.3)
Standards	IEC 61851-1 and OCPP context	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	Digital coordination and scope decisions	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	Risk register and mitigations	SO2/SO4	Ch. 3 (3.7)
Security considerations	Authentication, validation and authorization concepts	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	Module interfaces and complete workflow	SO1/SO2	Ch. 3 (3.5)
Project planning and management	Milestone plan, roles and budget structure	SO5	Ch. 4 (4.8)
Individual contribution	Module 1 ownership and reflection	SO1–SO7	Contribution Statement + Ch. 4 / Ch. 5